

Guess Market

Exercise 2 - JavaFX Prediction Market Application

README and Submission Documentation

Bonuses Implemented

The bonuses below were implemented for this exercise. Full details, including exactly how to switch each one on, are in the [Bonuses - Details and How to Activate](#) section later in this document.

Bonus	Points	Where to find it
Bonus 1 - Skin Switching	5 (up to 100)	SKIN dropdown, top-right of the toolbar
Bonus 2 - UI Animations	5 (up to 100)	Animations checkbox, top-right of the toolbar
Bonus 3 - Price and Balance Charts	8 (up to 100)	Automatic - inside any event's detail view, and on every user's page
Bonus 4 - Create a New Event	10 (above 100)	+ Create Event button on a selected user's page (Users tab)

Submitted By

Name	Nave Kluska
ID	324168160
Email	navekluska@gmail.com

GitHub Repository

<https://github.com/NaveKluska/GuessMarket>

Running the Application

- Unzip the submission archive to any folder, including one whose path contains spaces.
- Run `run.bat` (double-click it, or run it from a command prompt or PowerShell).
- When the window opens, click `Load File...` and choose an Exercise 2 events XML file via the file picker.

The batch file launches the application with everything it needs already bundled - the JavaFX SDK, JAXB, and the application's own jars - so nothing needs to be separately installed on the reviewer's machine.

A note on the `-Dprism.order=sw` flag

During development I hit a recurring crash inside JavaFX's own rendering pipeline: a `NullPointerException` in `com.sun.prism.d3d.D3DTexture`, thrown while rendering CSS rounded corners under Windows' Direct3D (D3D) graphics backend. This sits entirely inside JavaFX's internal rendering code - the application makes no direct use of any GPU or image resources (no `ImageView`, `Canvas`, or `snapshot()` calls anywhere in the codebase), so the fault is not reachable from our own code.

Passing `-Dprism.order=sw` switches JavaFX to its Software (SW) rendering pipeline instead of Direct3D, which avoids the faulty code path entirely. I verified there is no visible difference in the UI and no noticeable slowdown, since this application is form- and table-driven rather than graphics-intensive. The flag is already built into `run.bat`, so the reviewer does not need to do anything extra for it to take effect.

System Overview

This project implements Guess Market as a JavaFX desktop application, built directly on top of the Exercise 1 engine. It supports two trading mechanisms - LMSR (Logarithmic Market Scoring Rule) and Order Book (a two-sided limit order book with optional share minting) - together with multi-user accounts and balances.

Module structure

- `guess-market-dto` - plain, immutable data-transfer objects that cross the engine and UI boundary. The UI never touches engine internals directly.
- `guess-market-engine` - the core, UI-agnostic engine: the models (`Event`, `LmsrEvent`, `OrderBookEvent`, `User`, the order book types), XML parsing (JAXB), commission calculation, and all business rules. This module has no dependency on JavaFX and does not know who is calling it - the exact same engine also drives the Exercise 1 console application, unchanged.
- `guess-market-javafx` - the JavaFX UI: `MainController` and the FXML and CSS that present the engine's data and turn user actions into engine calls.
- `guess-market-console` (Exercise 1) is retained in this repository for continuity, but is not part of this Exercise 2 submission.

Design Choices and Assumptions

Where the specification left a decision open, the choices below were made deliberately, and are explained here as the specification requests.

Filters as dropdowns rather than toggle buttons

The Events tab's Method, Status, and Commission filters are implemented as ComboBoxes, each with an explicit All option, rather than as toggle buttons. This satisfies the requirement to view every event when no specific filter is wanted, while keeping the filter bar compact for a set of single-select, mutually exclusive choices.

Being blocked is tied to a negative balance, not permanent

A user becomes blocked exactly when their balance goes negative, and is automatically un-blocked the moment a credit - a payout, a sale, a subsidy refund, and so on - brings their balance back to zero or above. I read being blocked as a live consequence of a negative balance rather than a one-way status: the alternative, a user locked out forever after a single trade even once the market later pays them enough to cover it, did not seem to be the intended behaviour.

A blocked Market Maker may close their event, but not open one

Opening an event still requires the Market Maker not be blocked, since opening spends the MM's own money (the LMSR subsidy, or the Order Book's initial share allocation) and a blocked MM cannot be trusted to cover it. Closing is treated differently: the Market Maker is the only user who can ever close their own event, so refusing that action would permanently strand every participant's holdings and cash in an event that could never be resolved. A blocked MM is therefore still allowed to close, and thereby resolve, their event.

Trading happens from the Users tab

Per the specification, buying, selling, opening, and closing all happen from the Users area once a user is selected. The Events tab is a read-only, market-wide view of every event's current state, used for browsing rather than acting.

An Order Book sell order reserves shares rather than removing them

Placing a sell order keeps the shares in the seller's own holdings - and therefore still eligible for a payout if the event closes before the order fills - right up until a real trade happens. The engine separately tracks how many shares a user has already offered across their open asks, purely to stop them offering more than they actually hold.

Commission and trade-history labels are always from the viewer's perspective

On a user's own page, Your commission and Your trade history refer specifically to that user's own figures; the market-wide totals are shown separately on the read-only Events tab, so the same number is never shown under two different labels depending on who is looking at it.

A created event (Bonus 4) is not persisted

An event created through the Create Event bonus behaves exactly like one loaded from the file - it starts inactive and must be opened by its creator before trading can happen - but

since this exercise has no save and load, it exists only for the current session and is lost if a new file is loaded.

New Classes Documentation

A general, non-exhaustive overview of the main classes added for this exercise, grouped by module.

Engine - Order Book model

- [OrderBookEvent](#) - an [Event](#) subclass modelling a two-sided Order Book market: its per-option books, share holdings, mint policy, and base value d.
- [OrderBookMarket](#) and [OrderBook](#) - the matching engine. [OrderBook](#) does price-time-priority matching for a single option in isolation; [OrderBookMarket](#) coordinates one option's book against the others and applies the mint rule.
- [Order](#), [Fill](#), [Mint](#), [Holdings](#), [MarketQuote](#) - supporting model types: a resting order, a completed match, a completed mint, per-user per-option share counts, and the LAST, BID, ASK, MID and SPREAD statistics shown on screen.

Engine - supporting the bonuses

- [MarketMethodSpec](#) - a sealed interface ([Lmsr](#) and [OrderBook](#) records) carrying the method-specific settings needed to found a brand-new event (Bonus 4), keeping that call type-safe instead of one method taking every parameter of both market types.
- [BalancePoint](#) and [PricePoint](#) - one recorded balance or traded-price reading, used to build the Bonus 3 charts.
- [Amounts](#) - formats a monetary amount to two decimals in a fixed locale, so every error message shows a clean figure like 62.01 rather than a raw double.

DTOs

- [EventSummaryDTO](#) and [EventDetailsDTO](#) ([LmsrEventDetailsDTO](#), [OrderBookEventDetailsDTO](#)) - everything the Events tab needs to list, and then drill into, an event.
- [UserSummaryDTO](#) and [ParticipantHoldingDTO](#) - a user's own summary, and one participant's position within a specific event.
- [OptionDTO](#), [OptionBookDTO](#), [OrderDTO](#), [TransactionDTO](#) - an LMSR option's price and shares, an Order Book option's statistics and resting orders, one resting order, and one completed trade.
- [ChartPointDTO](#) - one timestamp-and-value point, shared by both Bonus 3 charts.

JavaFX

- [MainController](#) - the single controller behind [main.fxml](#). Builds every screen the user sees - the Events and Users tabs, the LMSR and Order Book detail views, the trading forms, the Create Event dialog, and the charts - directly from the DTOs the engine returns, and turns user actions back into engine calls.

- `Main` - the application entry point: wires the engine, loads `main.fxml`, and shows the window.

Bonuses - Details and How to Activate

Bonus 1 - Skin Switching

Three colour schemes - Classic (the default), Midnight (dark), and Sunset (warm, light) - each changing the entire background, every button, and the font family and size of every label on screen.

How to activate: a SKIN dropdown sits in the top-right of the toolbar, next to the Animations toggle. It starts on Classic, as the exercise requires; choose Midnight or Sunset to switch instantly. Choosing Classic again removes the skin entirely.

Bonus 2 - UI Animations

Three short animations, each well inside the two-second limit: the detail pane fades and eases in when an event or user is opened; a brief pulse confirms a trade just landed; the winner banner grows and fades into place when an event is resolved.

How to activate: an Animations checkbox sits in the toolbar. It starts unchecked, as required; check it to turn all three animations on. With it off, nothing is even constructed - there is no hidden cost to leaving it disabled.

Bonus 3 - Price and Balance Charts

Every LMSR and Order Book event detail view shows a Price over time chart, one line per option, plotted against trade number. Every user's own page shows an Account balance over time chart, plotted against clock time.

How to activate: nothing to switch on - these appear automatically once there is data to show them (at least one trade for the price charts, at least one balance-changing action for the balance chart).

Bonus 4 - Create a New Event

Any user can found a brand-new event - LMSR or Order Book, with its own name, description, options, and commission - and becomes its Market Maker. The new event then behaves exactly like one loaded from the file: visible on both tabs, starting inactive, and requiring its creator to open it (paying the usual subsidy or initial allocation) before trading can happen.

How to activate: select any user on the Users tab and click the + Create Event button beneath their balance. The button is disabled while the selected user is blocked.